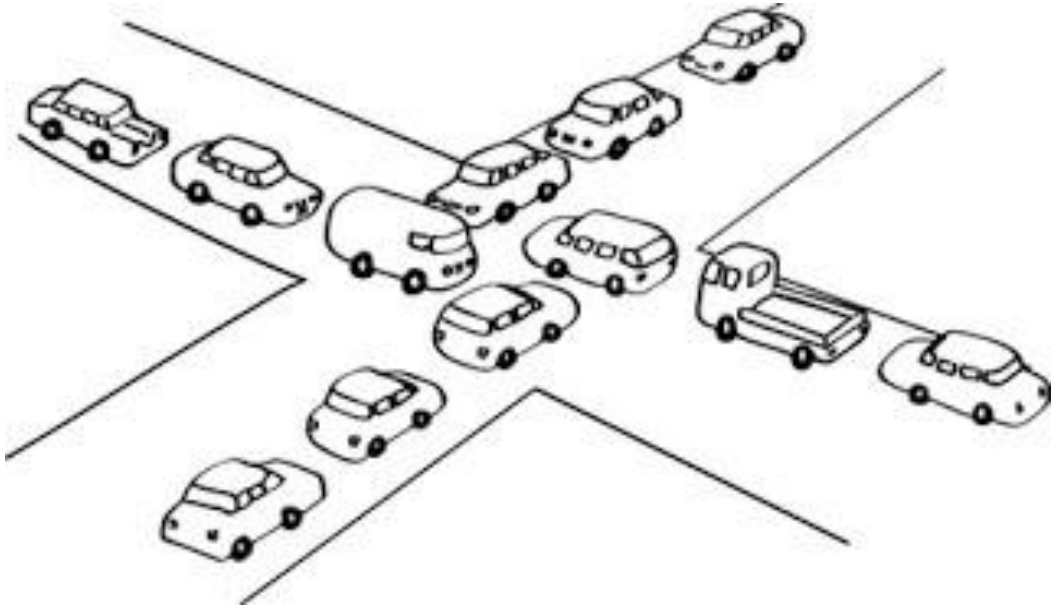
İşletim Sistemleri
(Operating Systems)



Kilitlenme (Deadlock)

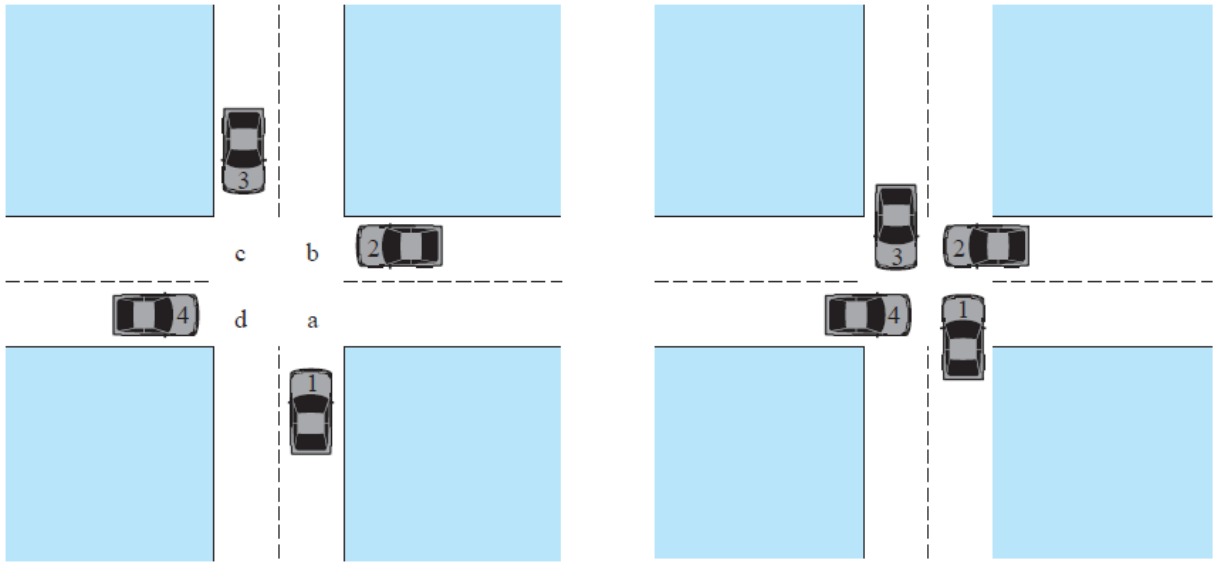
Kilitlenme (Deadlock)

Bilgisayardaki kilitlenmeler trafikteki kilitlenmelere benzer.

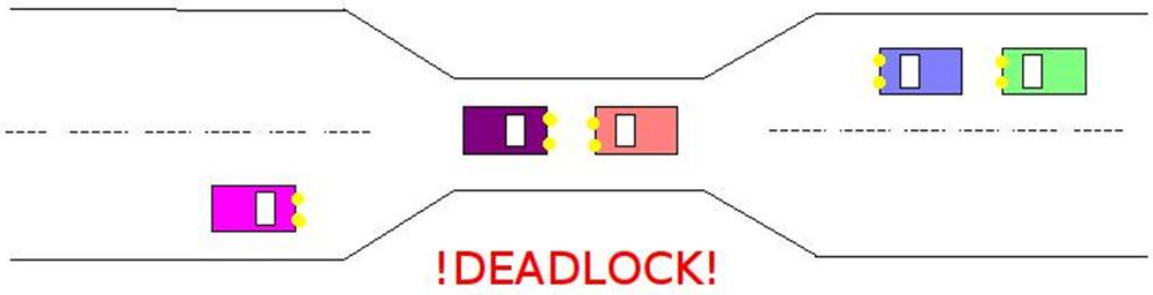
Örneğin, bir dörtyol kavşağına gelindiğinde uygulanması gereken kural sağdan gelene yol vermeyi belirtir.

Ancak eğer bu kavşağa aynı anda 4 araba gelirse ve hepsi de geçmeye çalışırsa kilitlenme olacaktır.

Yada, tek şeritli yolun iki araç tarafından da kullanılmak istenmesi sonucunda yolun tıkanması gibi.



(a) Deadlock possible (b) Deadlock

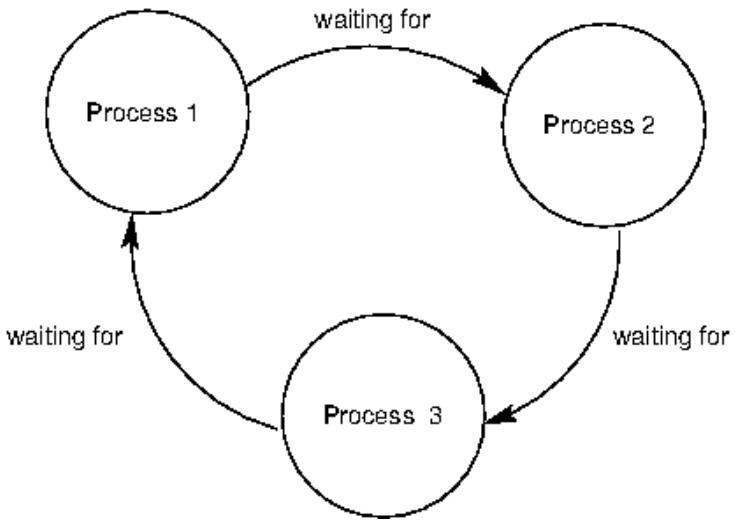


Kilitlenme (Deadlock)

- Bilgisayar sistemleri, aynı anda yalnızca bir yada birkaç process tarafından kullanılabilen donanım yada yazılım temelli bazı kaynaklara sahiptir.
- Çok process'li çalışma ortamlarında process'ler sonlu sayıdaki kaynakları kullanmak için yarışırlar.
- **Kilitlenme (deadlock), bekleme (waiting) durumundaki process'lerin hazır (ready) durumuna geçebilmek için diğer process'lerin sağlayacağı koşulu karşılıklı beklemelerine verilen addır.**
- Kilitlenme, genelde process'lerin mevcutta kullandıkları kaynakları serbest bırakmadan, başka process'lere tahsis edilmiş olan kaynakları da kullanmak istedikleri durumlarda gerçekleşir.

Kilitlenme (Deadlock)

- Kilitlenme iki yada daha çok process'in karşılıklı olarak hiç bir zaman gerçekleşmeyecek koşulları beklemeleriyle oluşan bir durumdur.
- Her process yalnız bir başka process'in sağlayacağı bir koşulu beklerse ve bu processler de kapalı bir zincir oluşturursa, kilitlenme meydana gelebilir.



Kilitlenme (Deadlock)

- Birlikte çalışan process'ler (cooperative processes) programlanırken tüm process'ler birlikte düşünülerek tasarım yapılır.
- Multithread uygulama geliştiriciler de kilitlenme (deadlock) olasılığına dikkat etmelidir.
- Olası kilitlenme durumları tasarım ve gerçekleştirim aşamasında bulunup ayıklanmalıdır.
- Bu nedenle, işletim sistemlerinde, process'ler arası senkronizasyona yönelik programlama hatalarından dolayı oluşabilecek kilitlenmelere karşı herhangi bir önlem öngörülmez.

Kilitlenme (Deadlock)

- Ana bellekte **200KB**'lık boş yer olduğunu varsayınız. İki process aşağıdaki sırayla ana bellekten istekte bulunurlar.

P1	P2
…	…
80 Kb istek √	70 Kb istek √
…	…
60 Kb istek	80 Kb istek

- Her iki process'in de ilk istekleri bellek tarafından karşılanınca geriye sadece 50 Kb'lik boşluk kalacaktır.
- Bu durumda her iki process de ikinci isteğini alamayacağı için kilitlenme olacaktır.

Kilitlenme (Deadlock)

- Bu gibi durumda kilitlenmeyi önlemek için process'lerin istekleri işlem sırasına göre yapılabilir.

P1	P2
…	…
80 Kb istek √	70 Kb istek
…	…
60 Kb istek √	80 Kb istek

- İlk olarak P1'in istekleri karşılanır. P1 işini bitirince bellekten silinecektir.
- Daha sonra da P2'nin istekleri karşılanır ve kilitlenme ortadan kaldırılır.

Kilitlenme (Deadlock)

- Bir sistemde, 3 tane CD WR sürücüsü olsun. 3 process bu kaynakları kullanırken, her process diğer CD WR sürücülerden birisine tekrardan istek yaparsa kilitlenme oluşur.

- Bir sistemde, 1 yazıcı ve 1 DVD sürücü olsun.
- P0 process'i yazıcıyı, P1 process'i DVD sürücüyü tutarken, P1 process'i yazıcıya ve P0 process'i DVD sürücüye istek yaparsa kilitlenme oluşur.

Kaynak Tahsis Grafı

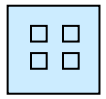
- Process’lerin istekleri ve kaynakların process’lere atanması **kaynak tahsis grafı** ile gösterilirler.
- Bir kaynak kategorileri kümesi {R1, R2, R3,. . ., Rn},
- Kaynaklar grafta **kare** şeklinde gösterilir.
- Her kaynak türü Ri'nin Wi örnekleri (instances) vardır.
- Kaynak gösterimi içindeki noktalar kaynağın örneklerini (instances) ifade eder, yani kaynağın kaç process’e atanabileceğini gösterir.

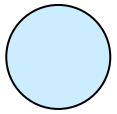
Kaynak Tahsis Grafı

- Bir process dizisi, {P1, P2, P3,. . ., Pn},
- Process'ler grafta **daire** şeklinde gösterilir.
- Bir process'ten kaynağa doğru giden ok, process'in kaynak için istek yaptığını gösterir.
- Kaynaktan process'e doğru gösterilen ok, kaynağın process'e tahsis edildiğini (atandığını) gösterir.

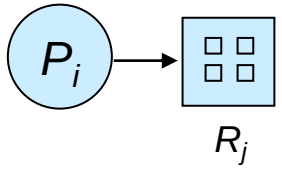
- Her process kaynakları aşağıdaki gibi kullanır:
- **Kaynak isteği (request)**
- **Kaynak atama (allocate)**
- **Kaynağın serbest bırakılması (release)**

Kaynak Tahsis Grafı

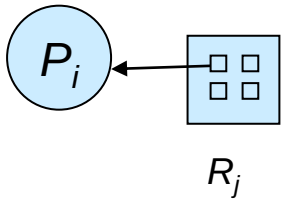
- Kaynak
- 4 örnekli (instances) kaynak (W_4) 

- Process 

- *P_i process'i R_j kaynağından bir örnek talep ediyor*



- *R_j kaynağının bir örneği P_i process'ine atanmış*



Kaynak Tahsis Grafı

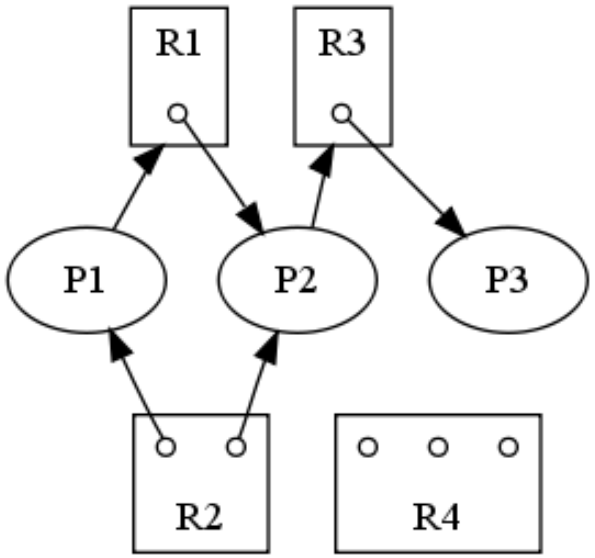
- P1, P2 tarafından tutulan bir kaynak talebinde bulunuyor.
- Hiçbir process R4 kaynağını talep etmiyor.

Kaynak atama (allocate)

R2 kaynağı P1 ve P2 ye atanmış.
$R_2 \rightarrow P_1 \;\; R_2 \rightarrow P_2$

R1 kaynağı P2 ye atanmış.
$R_1 \rightarrow P_2$

R3 kaynağı P3 e atanmış.
$R_3 \rightarrow P_3$



Kaynak talebi (request)

P1 process'i R1 için istekte bulunuyor.
$P_1 \rightarrow R_1$

P2 process'i R3 için istekte bulunuyor.
$P_2 \rightarrow R_3$

Bölüşülür ve bölüşülemez kaynaklar (Preemptable and non-preemptable resources)

- Kilitlenmeler sistem kaynaklarının process'ler arasında paylaşım zorunluluğundan ortaya çıkar.
- Bilgisayarda bulunan kaynaklar iki kategoriye ayrılabilir.

- **1. Bölüşülür-geri alınabilir kaynaklar (preemptable resources)**
- **2. Bölüşülemez-geri alınamaz kaynaklar (non-preemptable resources)**

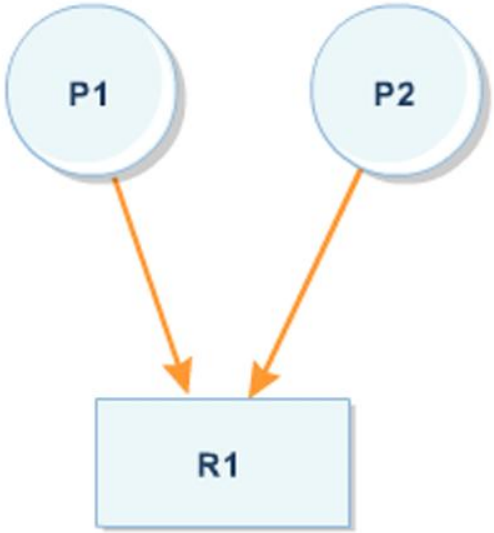
Bölüşülür kaynaklar (preemptable resources)

- Process'ler tarafından eşzamanlı biçimde paylaşılabilen kaynaklar bölüşür kaynaklardır (preemptable resources).

- İşlemci (CPU), ana bellek bölüşülür kaynaklardır.

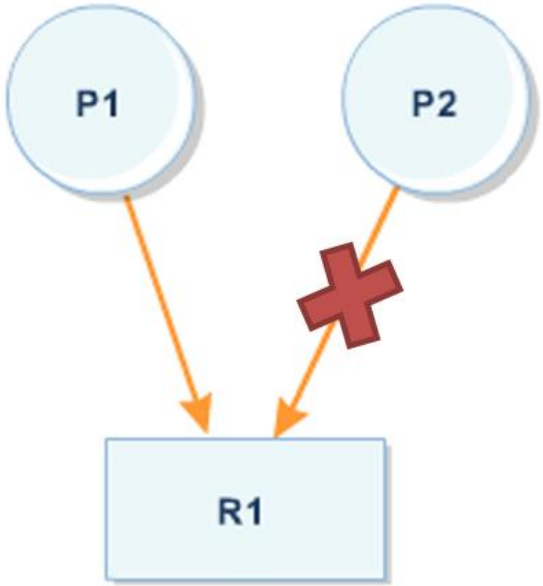
- **CPU ve ana bellek gibi <u>bölüşülebilir kaynaklar</u> işletim aşamasında bir process'ten geri alınıp başka bir process'in kullanımına verilebilir.**

- G/Ç aygıtları, dosyalar ve veritabanı kütükleri **<u>okuma modunda</u>** bölüşülür kaynaklardır.



Bölüşülemez kaynaklar (nonpreemptable resources)

- Process'ler tarafından ardıl biçimde kullanılması zorunlu olan kaynaklar bölüşülemez kaynaklardır (nonpreemptable resources)
- Bu kaynaklar istem üzerine aynı anda en çok bir process'e atanan, sınırlı (sonsuz olmayan) bir süre sonunda serbest bırakılan ve başka bir process'in kullanımına verilen kaynaklardır.
- **Bölüşülemez kaynakların kullanımları sona ermeden bir process'den geri alınarak diğer bir process'e atamaları yapılamaz.**



Bölüşülemez kaynaklar (nonpreemptable resources)

- Örneğin, bir process CD yazıcıyı kullanıyorsa CD yazıcının kullanan process'in elinden alınması oluşan CD nin bozulmasına yol açar. CD yazıcı bölüşülemez (nonpreemptable resources) bir kaynaktır.

- Örneğin; yazıcı (printer) bir yazma işlemindeyken, işletim sistemi yazıcıyı bu process'den alıp başka bir process'e veremeyeceği için yazıcı bölüşülemez bir kaynaktır.

- Bölüşülemez kaynaklara birden fazla process tarafından eşzamanlı erişim yapılmamalıdır (mutex lock, semafor veya monitörler ile engellenir).

Bölüşülemez kaynaklar (nonpreemptable resources)

- Bölüşülemez kaynaklarla ilgili kilitlenmeler, bu türdeki kaynakları paylaşan belirli sayıda process'le sınırlı kalır.

- Process'lere atanan kaynakların, process'ler istemedikçe alınamaması kilitlenmeleri oluşturan bir koşuldur.

- Atanan kaynakların, process'lerin istemleri dışında da sistemce geri alınmasının sağlanması, kilitlenmelerden korunmada akla gelen bir yöntem olabilir. Ancak bu yöntem bölüşülemez kaynaklar için uygulanamaz.

Kilitlenme için Gerekli Koşullar (Conditions for Deadlock)

- Processler arasında kilitlenme, yalnızca paylaşılamayan tek örnekli (instance) kaynaklar için aşağıdaki dört koşulun tümü aynı anda sağlandığında gerçekleşir.

1. **Karşılıklı dışlama durumu (mutual exclusion condition)**
2. **Sahiplenme ve bekleme durumu (hold and wait condition)**
3. **Bölüşülemez-Geri alınamaz kaynak durumu (no preemption condition)**
4. **Döngüsel bekleme durumu (circular wait condition)**

Aslında dördüncü koşul, ilk üç koşulun potansiyel bir sonucudur.

1. Karşılıklı dışlama (mutual exclusion):

- Process'lerin karşılıklı dışlama davranışları içinde bulunmaları durumu;
- Bir kaynağın aynı anda yalnız bir process tarafından kullanılabilmesi **(aynı anda bölüşülememesi)** durumudur.

- İki veya daha fazla process veya thread'ın, aynı anda bir kritik bölge işlemi yürütmemesinin istendiği durumlarda karşılıklı dışlama özelliği kullanılabilir.

- Fakat, karşılıklı dışlamayı ortadan kaldırmak, hemen hemen tüm donanımsal kaynaklar için olanaksızdır.

2. Sahiplenme ve bekleme (hold and wait):

- Bir process diğer process'ler için gerekli olabilecek (veya olmayabilir) en azından bir kaynağı elinde tutmalı ve diğer process'ler tarafından tutulan ek bir kaynağa gereksinim duymalı ve bu kaynağa erişene kadar sahip olduğu kaynağı serbest bırakmayacak olmasıdır.

- Yani bir process bir kaynağa sahipken başka bir kaynak talebinde bulunduğu durumdur. Talep edilen kaynak ise başka bir process tarafından kullanılır durumdadır.

- Bir sistemde sahiplenme ve bekleme (hold and wait) durumunun oluşmaması için, bir process bir kaynağa istek yaptığında başka bir kaynağı tutmaması gerekir.

3. Bölüşülemez-geri alınamaz kaynak (no preemption):

- Bir process'e atanan kaynaklar, işletim sistemi tarafından process'in kendi isteği dışında elinden alınamıyorsa.
- Bir process'e atanan kaynakların yalnız bu process tarafından serbest bırakılması, işletim sisteminin bu kaynaklara kullanımda iken el koyamaması durumudur.
- Bir bölüşülemez kaynak, kendisini kullanan bir process'den istenilen her anda geri alınamaz. Ancak, process görevini tamamladıktan sonra bölüşülemez kaynağı serbest bırakabilir.
- Bu yüzden bölüşülemez kaynakların beklenmesi kilitlenmeye sebep olabilir.

- İşletim aşamasında kullanılan kaynakları gerektiğinde tekrar geri alan bir kaynak atama yönetimi uygulamak çoğu fiziksel kaynak için olanaksızdır.

4. Döngüsel bekleme (circular waiting):

- Process’lerin gereksinim duyduğu kaynakları, döngüsel olarak birbirlerinden beklemesi durumudur.

- Bir process’in elinde tuttuğu kaynaklardan bir yada daha çoğunun, bu process ile aynı döngüsel işletim zinciri içinde yer alan diğer process’ler tarafından istenmesi.

- Döngüsel olarak birbirini bekleyen en az iki process’le kilitlenme ortaya çıkar.

- İki veya daha fazla process dairesel bir zincir şeklinde kendinden önceki process’e ait olan kaynağı bekler.

4. Döngüsel bekleme (circular waiting):

- {P0, P1, ..., Pn} process'leri birbirini beklemektedir.

- P0 process'i P1'i, P1 process'i P2'yi, ..., Pn process'i de P0'ı beklemektedir (dairesel liste).

- Bu yüzden, kilitlenme varsa mutlaka döngüsel bir bekleme vardır;
- Ancak döngüsel bekleme varsa mutlaka kilitlenme olur da denilemez, çünkü bir kaynağın birden çok örneği (instance) varsa, bu örnekler farklı process'ler tarafından kullanılabilir ve döngü olmasına rağmen kilitlenme oluşmayabilir.

Kilitlenme için Gerekli Koşullar

- Bu dört koşulun tamamının aynı anda bulunması kilitlenmeyi oluşturur.
- Bu dört koşulun tamamı aynı anda bulunmazsa kilitlenme durumu oluşmaz.
- Kilitlenmeleri önlemek üzere bu koşullardan en az birinin oluşmasını engellemek yeterlidir.
- Ancak paralel process yürütümünün gerçekleştirildiği işletim sistemlerinde, olanaklar elverdiği ölçüde kilitlenme ile başedilmeye çalışılır.
- Kilitlenme ile başetmek, genelde üç değişik yaklaşımla ele alınır.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- Kilitlenme ile başa çıkmak için üç genel yaklaşım mevcuttur.

1) **Kilitlenmeyi Önleme (Deadlock Prevention)**
2) **Kilitlenmeden Sakınma (Deadlock Avoidance)**
3) **Kilitlenmeyi Algılama (Deadlock Detection)**

Kilitlenme önleme ve kaçınma, kilitlenmelerin oluşmasını önlemeyi amaçlayan proaktif yaklaşımlardır; kilitlenmeyi algılama ise kilitlenmeyi oluştuktan sonra tanımlayan ve çözen reaktif bir yaklaşımdır.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

1. Kilitlenmeyi Önleme (Deadlock Prevention)

- Kilitlenmelerin oluşması için 4 koşul belirtilmişti.
- **(mutual exclusion, hold and wait, nopreemption resource, circular wait)**

- **Kilitlenmeyi önleme, aynı anda bu dört koşuldan en az birinin oluşmasının engellenmesiyle gerçekleşir.**

- **Kilitlenmeyi önleme, işletime alınan process'lerin kaynak isteklerine sınırlandırma getirilerek sağlanır.**

- Bu durumda da, verimsiz kullanım ve düşük throughput ortaya çıkar.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **1. Kilitlenmeyi Önleme (Deadlock Prevention)**
- ***Karşılıklı dışlama (mutual exclusion);***
- Bazı kaynaklar doğası gereği paylaşılamaz olduğundan, karşılıklı dışlama tamamen ortadan kaldırılamaz.
- Zira bir sistemde yer alan kaynakların sayısı, işletilen process sayısından çok daha az olduğundan, bu kaynakların zorunlu paylaşımı karşılıklı dışlamayı da birlikte getirir.
- Bu durumda kalan diğer üç koşuldan birinin kırılması kilitlenmeyi önlemede fayda sağlayabilir.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **1. Kilitlenmeyi Önleme (Deadlock Prevention)**
- ***Sahiplenme ve bekleme (hold and wait)*** durumunu ortadan kaldırmanın iki yolu vardır:
- ***Sahiplenmeyi ortadan kaldırarak :*** Process, yürütme başladıktan sonra kaynak tahsisi beklemek zorunda kalmaması için ihtiyaç duyduğu kaynakları önceden belirtir.
- Process yürütülmeye başlamadan önce gerekli tüm kaynaklar tahsis edilebilirse, sahiplenme ve bekleme (hold and wait) durumu ortadan kalkar. Sorun: Kaynak verimliliği düşer, açlık (starvation) riski artar.
- ***Beklemeyi ortadan kaldırarak :*** Process'in, yeni bir istekte bulunmadan önce halihazırda elinde bulunan tüm kaynakları serbest bırakması gerekir.
- Zorluklar:
- Bir process komutları birer birer yürüttüğünden, yürütmeden önce gerekli tüm kaynaklar hakkında bilgi sahibi olamaz.
- Bir process'in halihazırda tuttuğu tüm kaynakların serbest bırakılması da sorunludur çünkü bunlar diğer process'ler tarafından kullanılamayabilir ve gereksiz yere serbest bırakılabilir. Bu durum düşük cihaz kullanımına yol açacaktır.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **1. Kilitlenmeyi Önleme (Deadlock Prevention)**
- ***Bölüşülemez-geri alınamaz kaynak (nopreemption resource);***
- Kilitlenme, bir process'in başladıktan sonra kaynağın geri alınamaması nedeniyle ortaya çıkar. Ancak, kaynak, kilitlenmeye neden olan process'ten geri alınabilirse kilitlenme önlenebilir.
- Fakat bu hiç de iyi bir yaklaşım değildir çünkü eğer process tarafından kullanılan bir kaynak geri alınırsa, o ana kadar yapılan tüm işler tutarsız hale gelebilir, bu durum veri bütünlüğünü tehlikeye atabilir.

- Bir yazıcının herhangi bir process tarafından kullanıldığı farzedelim. Yazıcıyı o process'den alıp başka bir process'e atarsak hem basılan tüm veriler tutarsız ve etkisiz hale gelebilir, hem de process'in kaldığı yerden tekrar baskıya başlayamaması performans düşüklüğüne ve verimsizliğe neden olur.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **1. Kilitlenmeyi Önleme (Deadlock Prevention)**
- Aslında "Karşılıklı dışlama", "sahiplenme ve bekleme" ve "bölüşülemez-geri alınamaz kaynak" koşulları pratikte ihlal edilemez.
- Sonuçta döngüsel beklemeyi (circular wait) ortadan kaldırmak, kilitlenmeyi önlemenin tek pratik yoludur.

- **Döngüsel bekleme, pratikte kaynakların sıralı (numaralandırılmış) tahsisi ile engellenebilir.**

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **1. Kilitlenmeyi Önleme (Deadlock Prevention)**
- Döngüsel beklemeyi (circular wait) ortadan kaldırmak için
- Her kaynağa **benzersiz bir numara** atanır (örneğin, disk=1, yazıcı=2, tarayıcı=3).
- Process'ler, kaynakları **artan sırada** talep etmelidir.
  - Önce küçük numaralı, sonra büyük numaralı kaynak istenir.
  - **Kural:** Bir process $\mathbf{R_2}$'yi talep edecekse, önce $\mathbf{R_1}$'i almalıdır (eğer $R_1 < R_2$).

- Process'ler kaynakları sırayla talep ederse, ters yönde bekleme oluşamaz.
- Çapraz kaynak talepleri engellenir.
- Dezavantajlar: Process'ler ihtiyaç duymasa bile küçük numaralı kaynakları almak zorunda kalabilir. Yüksek numaralı kaynaklar sürekli bekleyebilir. Açlık (Starvation) riski.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

2. Kilitlenmeden Sakınma (Deadlock Avoidance) :

- Kilitlenmeden sakınma (deadlock avoidance), kaynak tahsisinin güvenli bir şekilde yapılmasını sağlayarak sistemin asla kilitlenme durumuna girmemesini garanti eder.
- Bu yöntem, kaynak taleplerini dinamik olarak analiz eder ve sadece güvenli durum (safe state) sağlandığında tahsis yapar.
- Kilitlenmeden sakınmada, kilitlenme oluşmadan önce tahmin etmek ve sistemin güvensiz duruma gitmemesi amaçlanır.
- Kaynaklar dikkatli bir şekilde tahsis edilirse kilitlenmeyi önlemek mümkündür.
- Kilitlenmeden sakınmada, ihtiyaç duyulacak her türden maksimum kaynak sayısı sürecin başında belirtilir.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

2. Kilitlenmeden Sakınma (Deadlock Avoidance) :

- İlke olarak, bir process, yeni bir kaynak isteminde bulunduğunda bu kaynakla ilgili işletim sistemi kesimi devreye girer.

- İşletim sistemi process'in bu isteminin potansiyel bir kilitlenmeye neden olup olmayacağını inceler.

- Kaynak ataması, ancak işletim sisteminin incelemesi sonucu olumlu ise gerçekleştirilir.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

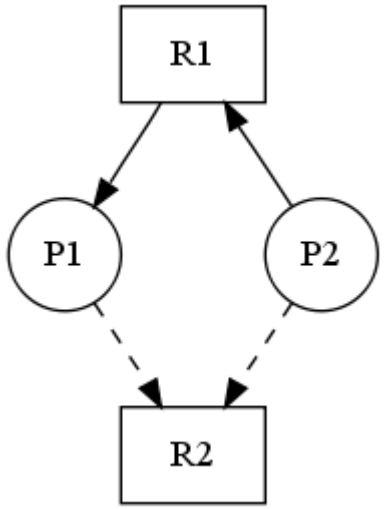
- **2. Kilitlenmeden Sakınma (Deadlock Avoidance) :**
- Sistem, ileride kilitlenme (deadlock) oluşmayacak şekilde kaynakları planlayarak tahsis eder.
- Tüm process'ler ihtiyaç duydukları kaynakları başlangıçta bildirirler, sistem döngüsel bekleme (circular wait) oluşmayacak şekilde kaynakları process'lere tahsis eder.
- Kilitlenmeden sakınma, process'leri çok uzun süre bloke edebilir.

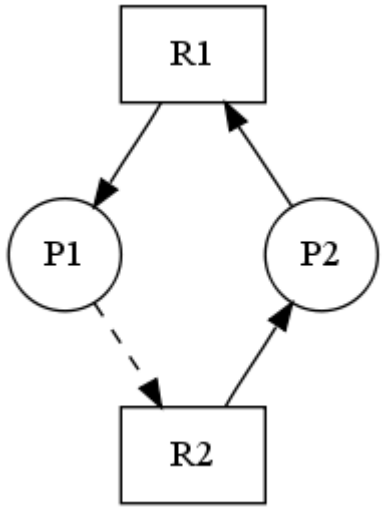
Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **2. Kilitlenmeden Sakınma (Deadlock Avoidance) :**
- **Güvenli Durum (Safe State)**
- Sistem, bir process sırası (safe sequence) bulabiliyorsa güvenlidir.
- Bu sırada her process, mevcut+serbest bırakılacak kaynaklarla tamamlanabilir.
- Eğer bir sistem, kaynakları process'lere belirli bir sırada **(safe sequence)** maksimum ihtiyaçları kadar atayabiliyorsa ve kilitlenme (deadlock) oluşmuyorsa bu durum **safe state** olarak adlandırılır.
- Eğer bir sistemde **safe sequence** varsa sistem **safe state** durumundadır.
- Güvenli bir sıra yoksa, sistem **güvensiz durumdadır (unsafe state).** Bu durumda kilitlenme olabilir, ama kesin değil.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **2. Kilitlenmeden Sakınma (Deadlock Avoidance) :**
- Çoğu sistemde, kaynakların her biri birer birer talep edilir.
- Sistem, bir kaynağın tahsisinin güvenli olup olmadığına karar verebilmeli ve güvende olduğunda kaynak tahsisi yapabilmelidir.





Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **2. Kilitlenmeden Sakınma (Deadlock Avoidance) :**
- Process'lere kaynak atanırken bunun ileride bir kilitlenmeye neden olup olmayacağını belirleyebilen bir algoritma 1965 yılında Dijkstra ve Habermann tarafından tanımlanmıştır.

- Process'lere, kilitlenme oluşturmadan kaynak atayabilme, bir bankerin iflas etmeden borç verebilmesine benzediğinden **Banker's Algoritması** olarak adlandırılmıştır.
- **Banker's Algoritması**, her kaynak talebini güvenli durum (safe state) testinden geçirir.

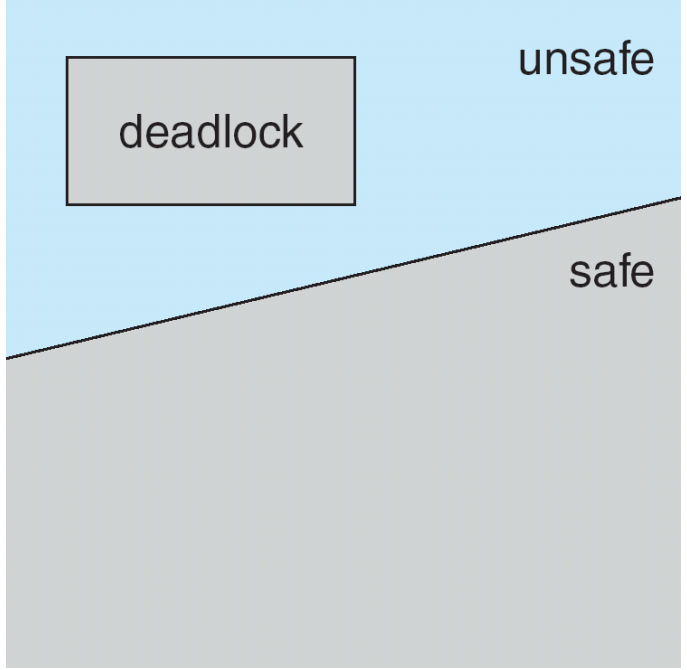
Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **2. Kilitlenmeden Sakınma (Deadlock Avoidance) :**
- **Banker’s Algoritması Avantajlar:**
- Dinamik çözüm: Kaynaklar esnek şekilde yönetilir.
- Yüksek kaynak kullanımı: Deadlock prevention'a göre daha verimli.

- **Banker’s Algoritması Dezavantajlar:**
- Yüksek hesaplama maliyeti: Her kaynak talebinde güvenlik testi gerekir.
- Önceden maksimum ihtiyaç bilinmeli: Gerçek sistemlerde bu her zaman mümkün değildir.

Banker’s Algoritması

- Belli bir sermayeye sahip bir bankerin iflas etmeden müşterilerine kredi imkanı tanıması esastır.
- Müşterinin tüm gereksinim duyduğu krediyi almadan, geri ödeme garantisi yoktur.
- Şayet bir miktar kredi talep edildiğinde diğer müşterilerin ihtiyaçlarının karşılanamaması riski varsa kredi verilmemektedir.
- Sistemin içinde bulunabileceği iki durum vardır; **güvenli (safe)** ve **güvenli olmayan (unsafe)** durum.



Banker’s Algoritması

- Banker's Algoritması, process’lerin kaynak taleplerini ve mevcut kaynak durumunu analiz ederek, bir kaynak talebi sonucunda sistemin güvenli (safe) bir durumda kalıp kalmayacağını değerlendirir.
- Banker’s Algoritması kullanılarak process’ler için güvenli şekilde çalışabilecek bir sıra **(safe sequence)** bulunursa kilitlenme olmaz demektir.
- Güvenli **(safe)** durumda, tüm process’lerin kilitlenme oluşturmayacak şekilde sonlanmasına olanak veren bir işletim şekli mevcuttur.
- Eğer böyle bir işletim şekli yoksa sistem için güvensiz **(unsafe)** bir durum söz konusudur ve kilitlenme olasılığı bulunur.
- Algoritma, bir kaynak talebiyle sistemin güvensiz bir duruma girmesini engellemek için talebi kabul etme veya reddetme kararı verir.

Banker’s Algoritması

- Banker's Algoritması, sistemin güvenli bir durumda kalmasını sağlamak için kaynak tahsislerini planlamak ve koordine etmek için kullanılan bir algoritmadır.
- Bu sayede kilitlenme (deadlock) durumları önlenir ve sistemdeki kaynaklar verimli bir şekilde kullanılır.
- Banker’s Algoritması çalışma prensibi, process’lerin başlangıçta talep edebilecekleri maksimum kaynak miktarını beyan etmelerine dayanır.
- Ardından, process’ler taleplerini yapar ve sistem, kaynakları taleplere göre tahsis etme ve güvenli bir durumu (safe state) koruma yeteneğini analiz eder.
- Eğer bir talep sonucunda sistem güvenli bir durumda kalıyorsa, talep kabul edilir; ancak güvensiz bir duruma yol açacaksa talep reddedilir.

Banker's Algoritması

- **Banker's algoritması** yönteminde;

➢ Kaynaklar (Resources vector) - (R)
➢ Boşta bulunan kaynaklar (Available vector) - (V)
➢ Talep edilen kaynaklar (Claim or Max request matrix) - (C)
➢ Atanan kaynaklar (Allocation matrix) - (A)

hakkında bilgi sahibi olunması gerekir.

Tüm kaynak taleplerinden (C), atanan kaynaklar (A) çıkarıldığı zaman **geriye kalan istek (Need)** sayıları **(Q)** bulunabilir. **Q = C – A**

<u>Talepleri kilitlenmeye yol açacaksa bir process'i başlatma.</u>

Banker's Algoritması

- *Bir sistemde **n*** process ve ***m*** farklı kaynak türü mevcut ise aşağıdaki vektörler ve matrisler tanımlanabilir:

- Kaynaklar (Resource) = $\mathbf{R} = (R_1, R_2, \ldots, R_m)$

- Boşta bulunan kaynaklar (Available) = $\mathbf{V} = (V_1, V_2, \ldots, V_m)$

- Talep edilen (Claim) = $\mathbf{C} = \begin{bmatrix} C_{11} & C_{12} & \ldots & C_{1m} \\ C_{21} & C_{22} & \ldots & C_{2m} \\ . & . & & . \\ C_{n1} & C_{n2} & \ldots & C_{nm} \end{bmatrix}$

- Atanmalar (Allocation) = $\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \ldots & A_{1m} \\ A_{21} & A_{22} & \ldots & A_{2m} \\ . & . & & . \\ A_{n1} & A_{n2} & \ldots & A_{nm} \end{bmatrix}$

Örnek-1 : Banker's Algoritması

- Aşağıda verilen bilgileri ve Banker's algoritmasını kullanarak sistemin process'ler için güvenli şekilde çalışabilecek bir sırası (safe sequence) mevcut mu? Sistemin güvenli durumda olup olmayacağına karar veriniz.

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Atanmalar (Allocation matrix - A)

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
0	1	1

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker’s Algoritması

- İlk olarak process’lerin geriye kalan isteklerini (Q) hesaplamak gerekir.
- Geriye kalan istekler (Need)= Talep edilen kaynaklar (Claim or Max request) - Atanan kaynaklar (Allocation)

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
0	1	1

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker's Algoritması

- Daha sonra geriye kalan istekler (Q) ve boştaki kaynaklar (V) kontrol edilerek çalışabilecek bir process var mı bulunması gerekir.
- Bu noktada P2 process'inin sadece R3 kaynağına ihtiyacı vardır ve bu ihtiyaç boştaki kaynakla karşılanabilir.

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
0	1	1

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker’s Algoritması

- P2 processine R3 kaynağı tahsis edilir.

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	1	0	0
P2	6	1	3
P3	2	1	1
P4	0	0	2

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
0	1	0

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker's Algoritması

- P2 process'inin istediği kaynaklar tahsis edildikten sonra, işlemini yapıp sonlanacak ve kullandığı tüm kaynakları serbest bırakacaktır.

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
6	2	3

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker's Algoritması

- Bu durumda 3 process'den herhangi biri için yetecek kadar kaynak bulunmaktadır. P1, P3 ve P4 process'lerinden herhangi bir tanesi çalışmaya başlayabilir. Sıra ile gidilecek olursa P1 kaynakları kullanıp çalışmaya başlayabilir.

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
6	2	3

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker’s Algoritması

- P1 process’inin istediği kaynaklar tahsis edildikten sonra, işlemini yapıp sonlanacak ve kullandığı tüm kaynakları serbest bırakacaktır.
- P1’in ardından P3 çalışabilir.

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
7	2	3

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker's Algoritması

- P3 işlemini yapıp sonlanacak ve kullandığı tüm kaynakları serbest bırakacaktır. Son olarak da P4 process'i çalışabilir.

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
9	3	4

Boştaki kaynaklar (Available vector - V)

Örnek-1 : Banker's Algoritması

- Böylece process'ler **P2, P1, P3 ve P4** sırası ile güvenli bir şekilde çalışabilir. Kilitlenme yoktur.
- Birden çok güvenli process sırası var. Tek bir güvenli sıra olduğu sürece, sistem kilitlenme durumunda değildir.

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	0

Talep edilen (Claim matrix - C)

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	0

Atanmalar (Allocation matrix - A)

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	0

Geriye kalan istekler (Need)
Q = C - A

R1	R2	R3
9	3	6

Kaynaklar (Resources vector - R)

R1	R2	R3
9	3	6

Boştaki kaynaklar (Available vector - V)

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **3. Kilitlenmeyi Algılama (Deadlock Detection)**
- Bir sistem kilitlenmeyi önleme (deadlock-prevention) veya kilitlenmeden sakınma (deadlock avoidance) yöntemlerini kullanmıyorsa, bir kilitlenme oluşma olasılığı her zaman vardır.
- Bunun için kaynakların process'lere tahsisi ve bekleyen tahsis talepleri hakkında bilgi tutulmalıdır.
- Daha sonra sistemin kilitlenme durumuna girip girmediğini belirleyecek bir **kilitlenmeyi algılama (deadlock detection) algoritmasına** ihtiyaç vardır.
- Kilitlenme algılama (deadlock detection), sistemin periyodik olarak kaynak tahsis grafiğini veya belirli veri yapılarını kontrol ederek kilitlenme durumunu tespit etme yöntemidir.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **3. Kilitlenmeyi Algılama (Deadlock Detection)**
- **Bu yöntem kullanıldığında, işletim sistemi process'lerin kaynak gereksinimlerini, istem/talep üzerine işletim aşamasında hiçbir kısıtlama getirmeden sağlar.**

- Yalnız process'lere atanan ve bunlar tarafından serbest bırakılan kaynakları izleyerek ortaya çıkan kilitlenmeleri algılamaya çalışır.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **Kilitlenmeyi Algılama (Deadlock Detection)**
- değişik yöntemlerle yapılabilir:
  - **Kaynak Tahsis Grafiği (Resource Allocation Graph - RAG) ile algılama**
  - **Bekleme Grafiği (Wait-for-Graph) ile algılama**
  - **Banker's Algoritmasına Benzer Tabanlı Algılama**

Kaynak Tahsis Grafı

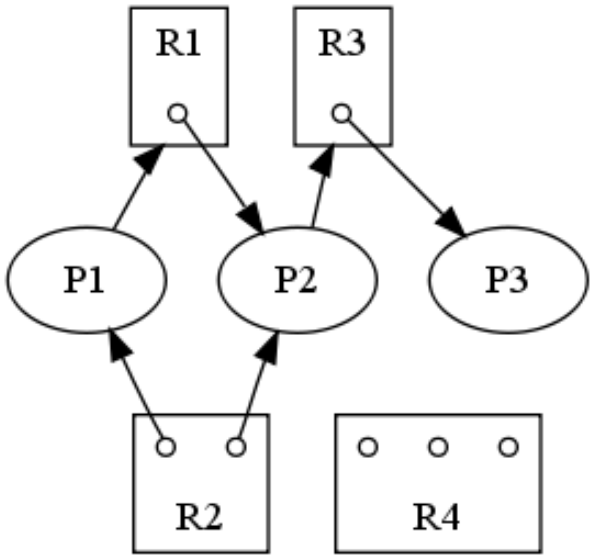
- P1, P2 tarafından tutulan bir kaynak talebinde bulunuyor.
- Hiçbir process R4 kaynağını talep etmiyor.

Kaynak atama (allocate)

R2 kaynağı P1 ve P2 ye atanmış.
$R_2 \rightarrow P_1 \;\; R_2 \rightarrow P_2$

R1 kaynağı P2 ye atanmış.
$R_1 \rightarrow P_2$

R3 kaynağı P3 e atanmış.
$R_3 \rightarrow P_3$



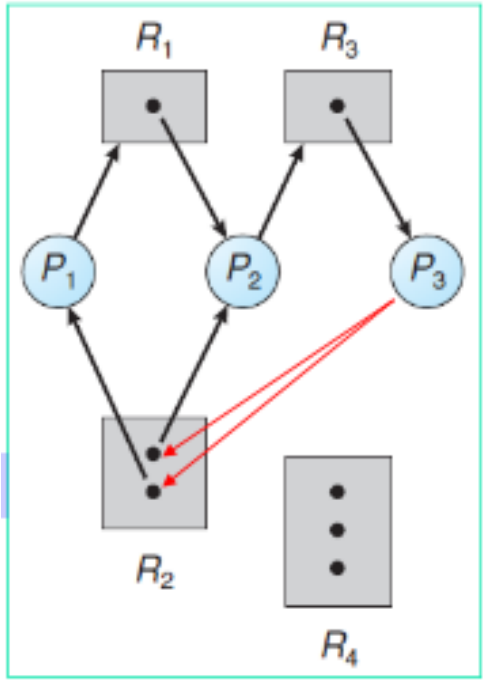
Kaynak talebi (request)

P1 process'i R1 için istekte bulunuyor.
$P_1 \rightarrow R_1$

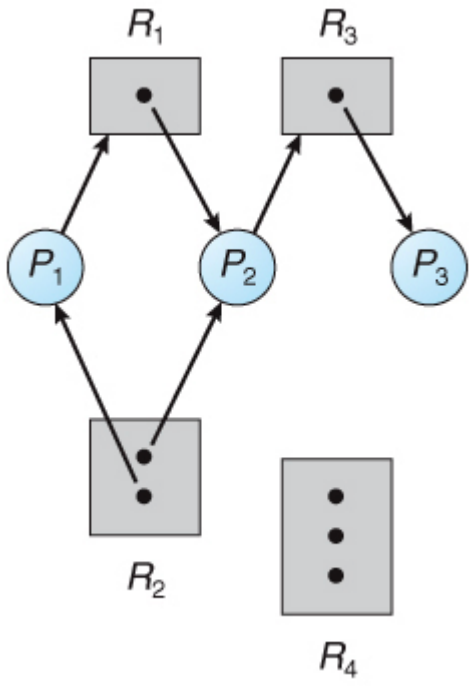
P2 process'i R3 için istekte bulunuyor.
$P_2 \rightarrow R_3$

Kaynak Tahsis Grafı

- Eğer graf üzerinde döngü yoksa kilitlenme (deadlock) yoktur, döngü varsa kilitlenme **olabilir**.
- Her kaynaktan sadece 1 örnek varken graf üzerinde döngü varsa, kilitlenme oluşur. Döngü içerisinde yer alan tüm process'ler kilitlenme durumundadır.
- Kaynakların birden fazla örneği olması durumunda, kilitlenme için döngü oluşması gereklidir ancak yeterli değildir.
- Döngü dışındaki bir process'e atanmış kaynak seçilebilir.



Kilitlenme olmadan kaynak tahsis grafı

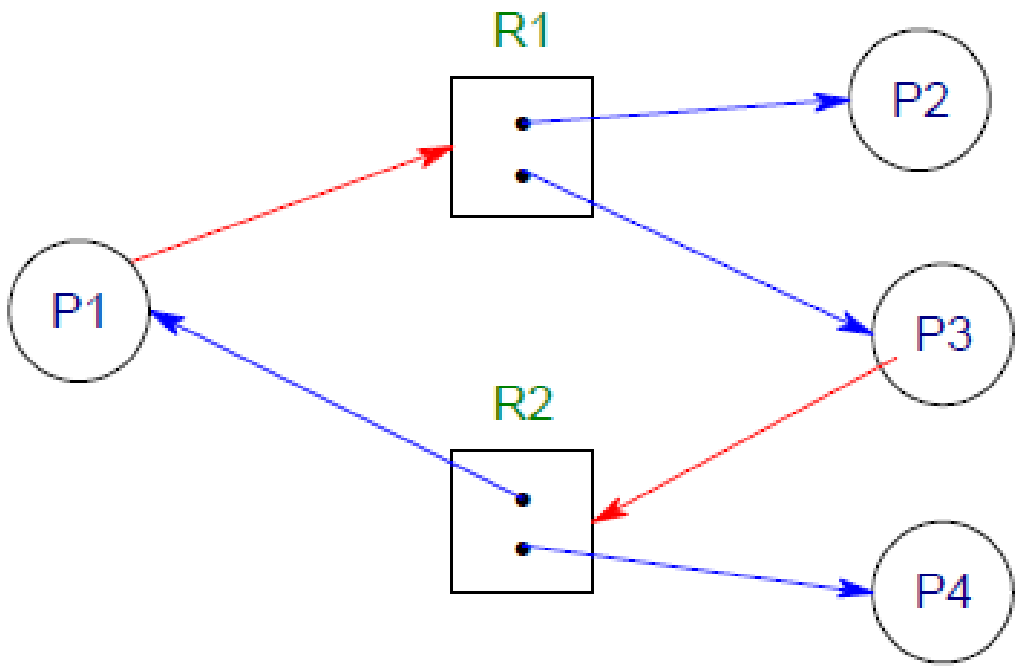


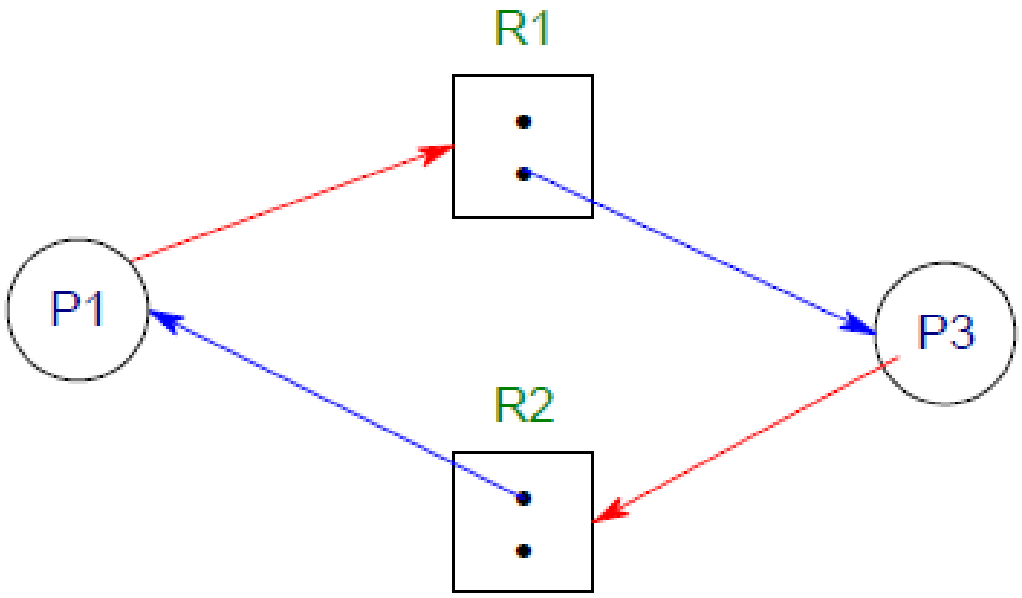
Şekilde P1, P2 ve P3 arasında kilitlenme (deadlock) durumu bulunmaz.

P3 sahip olduğu R3 kaynağını serbest bıraktığında, R3 kaynağı P2 process'ine atanır.

Kilitlenme olmadan kaynak tahsis grafı

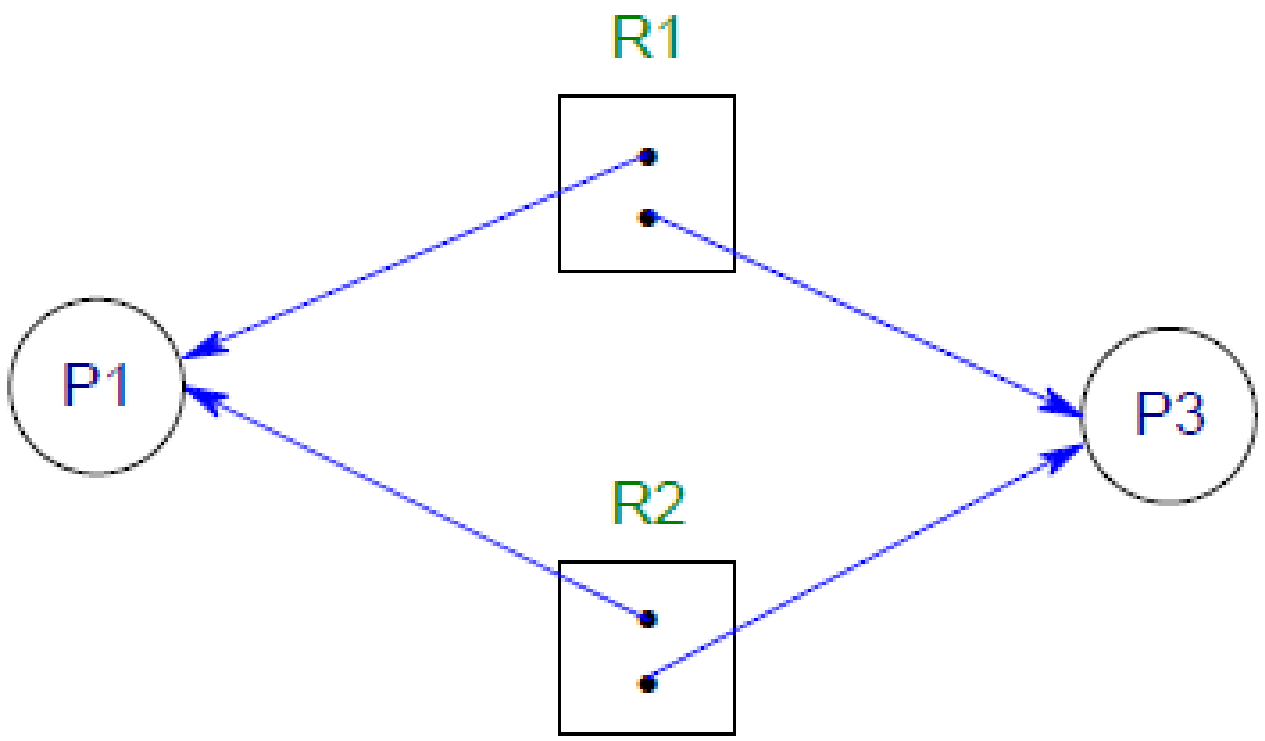
- P1 ve P3 process'leri boşta kaynağı olmayan R1 ve R2 kaynaklarını beklemektedir.
- Ancak P2 ve P4 istekte bulundukları tüm kaynaklara sahiptir.
- Bu durumda P2 ve P4 process'leri işlerini bitirince, kullandıkları kaynakları serbest bırakacaklardır. Bırakılan kaynakları diğer process'ler kullanabilir. **Dolaysıyla kilitlenme oluşmaz.**





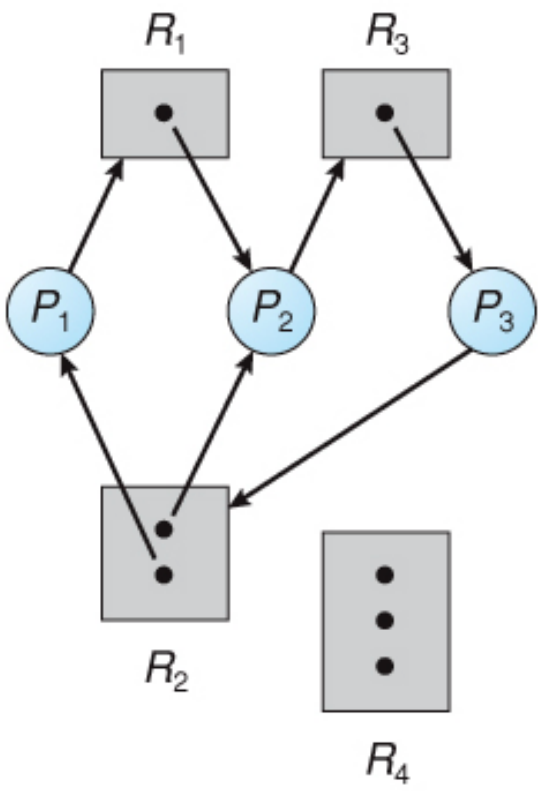
Kilitlenme olmadan kaynak tahsis grafı

- Aşağıdaki kaynak tahsis grafında döngü vardır, ancak kilitlenme (deadlock) yoktur.



Kilitlenme (deadlock) ile kaynak tahsis grafı

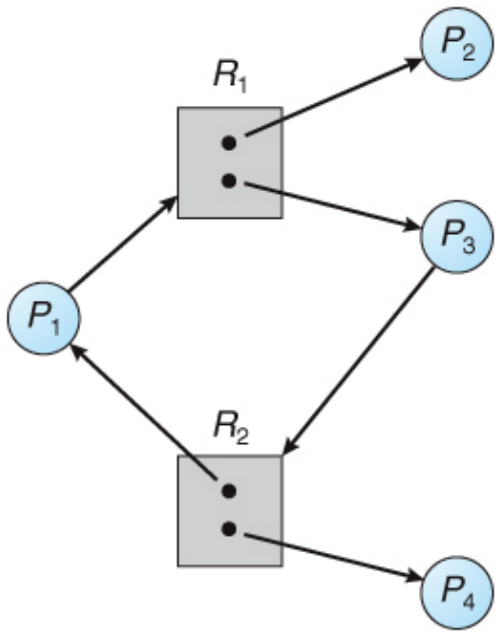
- Şekilde P1, P2 ve P3 kilitlenme (deadlock) durumundadır.

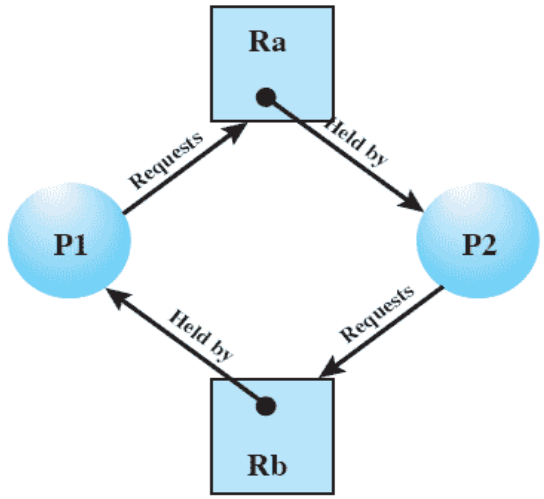


$$P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$
$$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$$

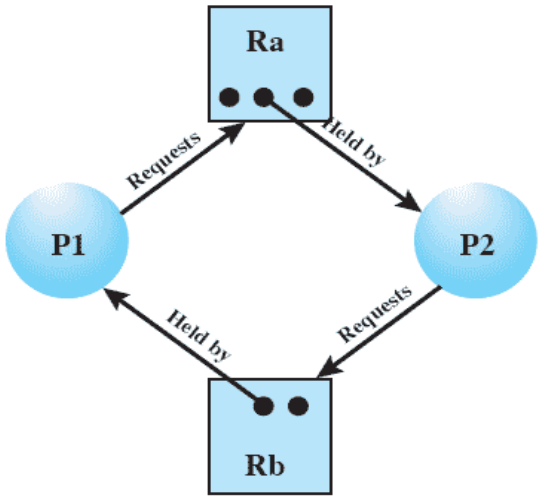
Kaynak tahsis grafında bir döngü varlığı

$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$





(c) Circular wait



(d) No deadlock

Bir kaynak tahsis grafı döngüler içeriyorsa ve her bir kaynak yalnızca tek bir process'e atanabiliyorsa, bir kilitlenme (deadlock) var demektir.

- Bir kaynak tahsis grafı döngü içermiyorsa, sistemde kilitlenme (deadlock) olmaz. (Döngü ararken, bunların yönlendirilmiş graflar olduğunu unutmayın.)
- Bir kaynak birden çok process'e atanabiliyorsa, kaynak tahsis grafında <u>bir döngü varlığı bir kilitlenme (deadlock) olasılığını gösterir ancak bir döngü kilitlenme olduğunu garanti etmez.</u>

Kilitlenme (Deadlock)

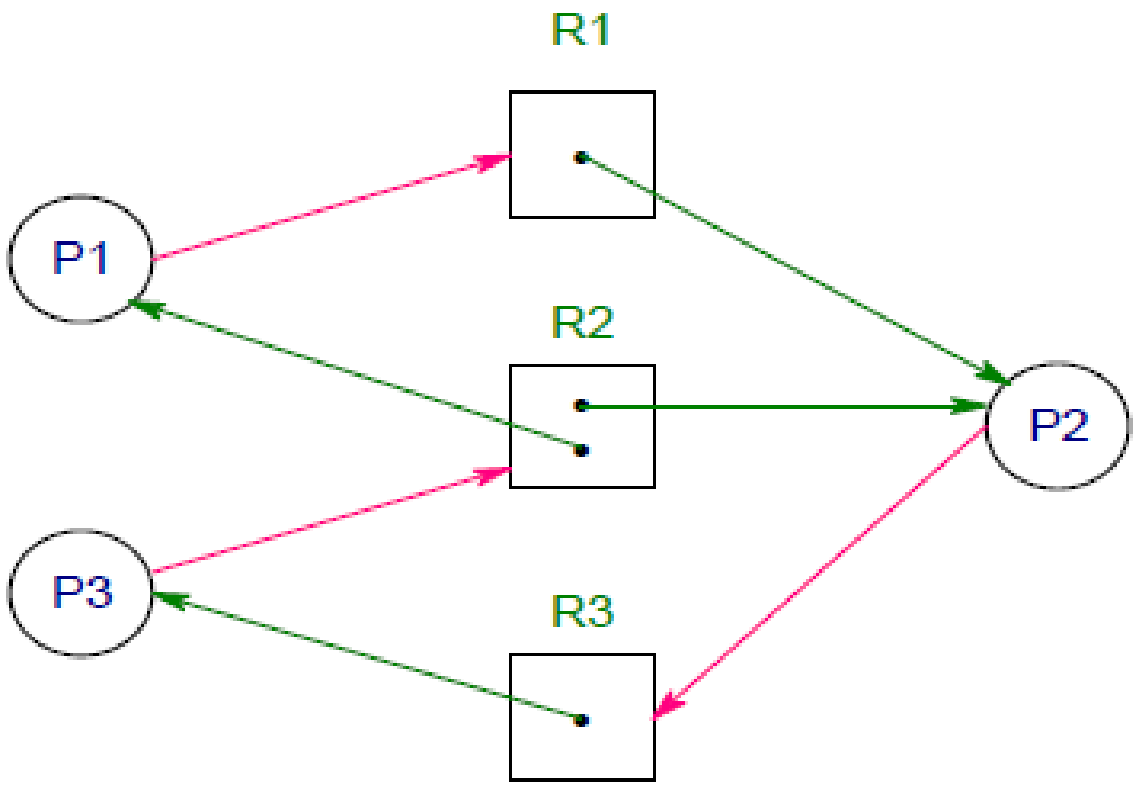
- **Örnek :**
- Aşağıdaki kümelerde P process'leri, R kaynakları, E ise process istekleri ile kaynak atamalarını belirtmektedir.
- P={P1, P2, P3}
- R={R1, R2, R3}
- E={P1→R1, P2 → R3, P3 → R2, R1 → P2, R2 → P2, R2 → P1, R3 → P3}
- R1 ve R3 kaynakları bir process'e, R2 kaynağı ise iki process'e atanabilmektedir.

a) Kaynak tahsis grafını çiziniz.

b) Kilitlenme durumu var mıdır, karar veriniz.

Kilitlenme (Deadlock)

- a) Kaynak tahsis grafı :
- E={P1→R1, P2 → R3, P3 → R2, R1 → P2, R2 → P2, R2 → P1, R3 → P3}



Kilitlenme (Deadlock)

b) Kilitlenme durumu var mıdır?

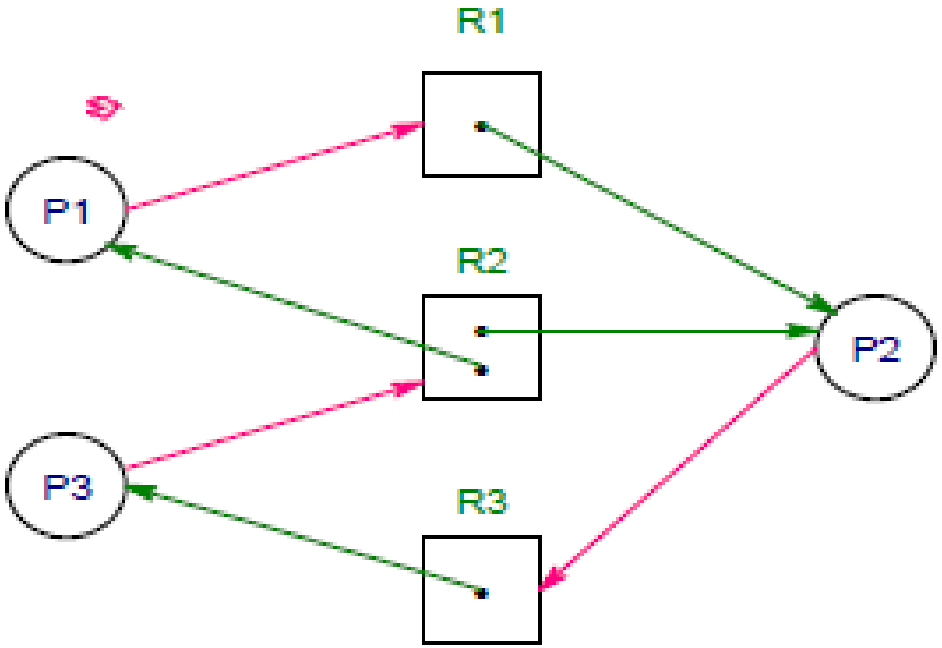
Kilitlenme vardır.

P1, P2 ve P3 process'leri kilitlenmiştir.

P1 process'i R1 kaynağını beklemektedir.

P2 process'i R3 kaynağını beklemektedir.

P3 process'i R2 kaynağını beklemektedir.



Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **Wait-for-Graph ;**
- Eğer kaynaklar tek bir örneğe (instance) sahipse,
- Kilitlenmeyi algılama, hangi kaynakların hangi process'lere atandığını gösteren kaynak tahsis graflarının (resource allocation graph) bir varyantı(çeşidi) olarak bilinen **wait-for graph** kullanarak yapılabilir.

- Wait-for-Graph bir döngü içeriyorsa, sistemin Kilitlenme durumunda olduğu söylenebilir. Grafta bir döngünün varlığı, kilitlenme için yeterli bir koşuldur.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

- **Wait-for-Graph ;**
- **wait-for graph**'daki bir döngü potansiyel bir kilitlenmeye işaret eder. Algoritma, derinine öncelikli arama (depth-first search) veya döngü tespit algoritmaları gibi çeşitli algoritmalar kullanarak grafdaki döngüleri arar.
- Bu yaklaşım nispeten basit ve etkilidir ancak birçok kaynağa sahip büyük sistemler için hesaplama açısından pahalı olabilir.

Kilitlenme ile başa çıkmak (Handling Deadlocks)

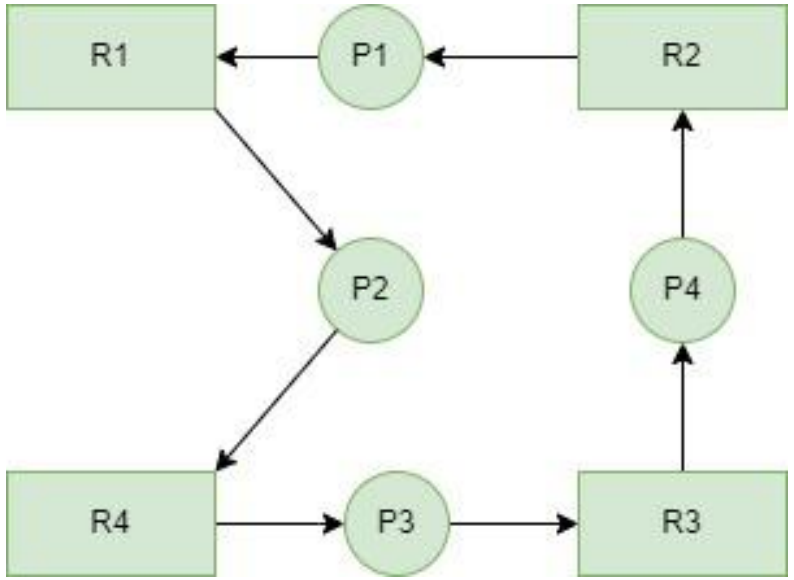
- **Wait-for-Graph ;**

- Resource Allocation Graph: Process'leri ve Kaynakları içerir.

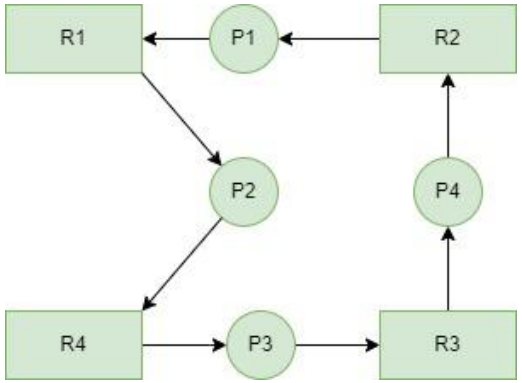
- Wait-for-Graph: Kaynak tahsis grafını, wait-for graph durumuna dönüştürürken kaynakları kaldırmak gerekiyor.
- Kaynak Tahsis Grafından (Resource Allocation Graph) dönüştürme sırasında Kaynaklar kaldırıldıktan sonra sadece Process'leri içerir.

Örnek-1:

- 4 Process (P1, P2, P3, P4) ve 4 Kaynak (R1, R2, R3, R4) içeren bir Kaynak Tahsisi Grafı verilmiş olsun.
- Wait-for-Graph tabanlı kilitlenme algılama algoritmasını kullanarak bir kilitlenme olup olmadığını bulunuz.

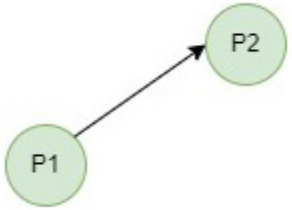


Örnek-1:

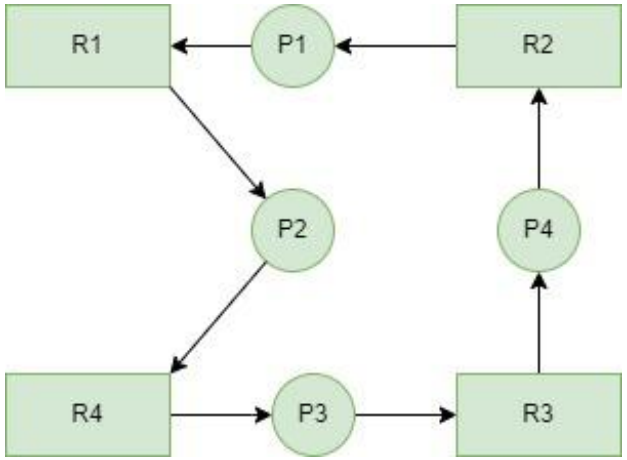


- Adım 1: Önce, Kaynak R1'i talep edip bekleyen Process P1 alınır, kaynak R1, Process P2 tarafından edinilir.

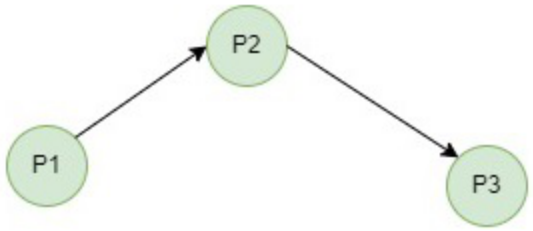
- Adım 2: Şimdi P1'den P2'ye bir yol olduğunu gözlemleyebiliriz çünkü P1, P2 tarafından alınan R1'i beklemekteydi. R1 kaynağını kaldırdıktan sonra grafın durumu aşağıdadır.



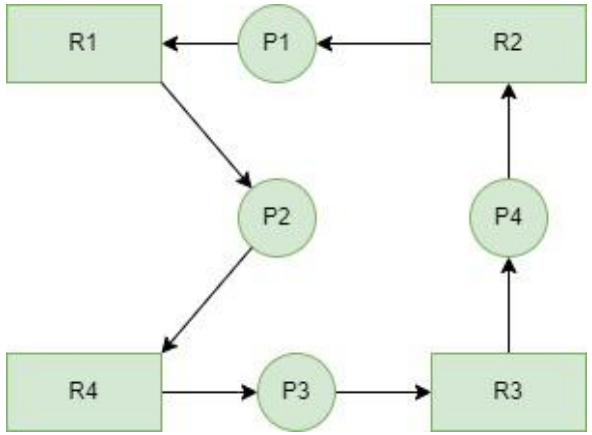
Örnek-1:



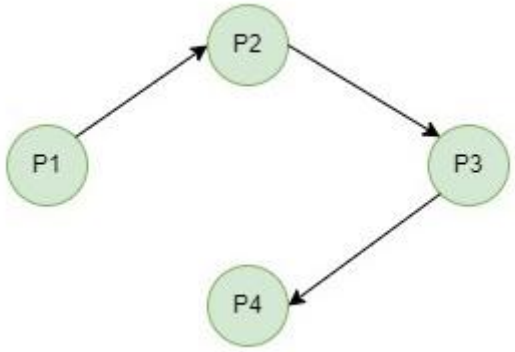
- **Adım 3:** P2, P3 tarafından elde edilen R4'ü beklediği için P2'den P3'e bir yol gözlemleyebiliriz. Bu nedenle, R4 kaynağını kaldırdıktan sonra P2'den P3'e bir yol oluşturulur.



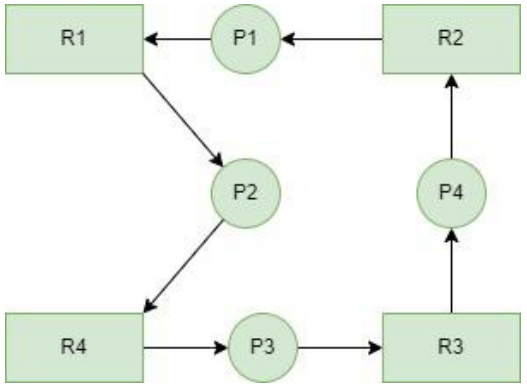
Örnek-1:



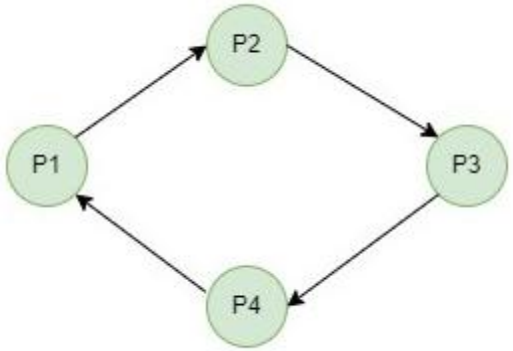
- **Adım 4:** P3, P4 tarafından elde edilen R3'ü beklediği için P3'den P4'e bir yol gözlemleyebiliriz. Bu nedenle, R3 kaynağını kaldırdıktan sonra P3'den P4'e bir yol oluşturulur.



Örnek-1:



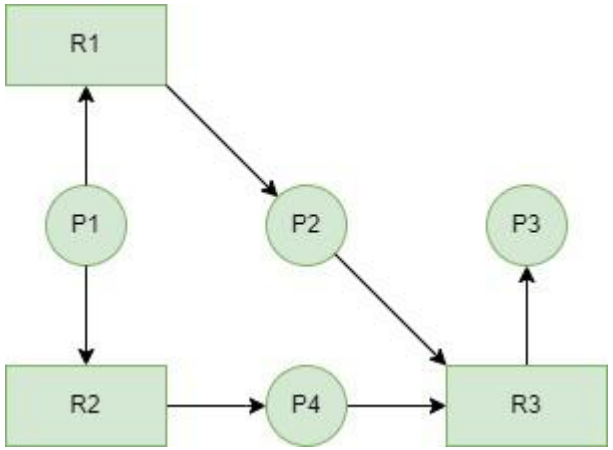
- **Adım 5:** Burada Process P4'ün P1 tarafından elde edilen R2'yi beklediğini görebiliriz. Sonuç olarak, Wait-for-Graph aşağıdaki gibidir:

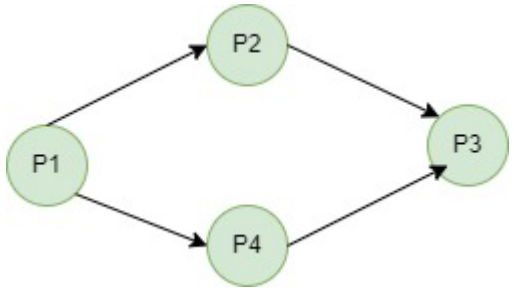


- Grafta kapalı bir döngü bulunmaktadır, yani burada sistemin kilitlenme durumunda olduğunu söyleyebiliriz.
- Kilitlenme çözümü, çoğu kez algılanan döngü içinde yer alan ve kilitlenmeye neden olan process'lerin yok edilmesi ya da tüm sistemin yeniden başlatılmasıdır.

Örnek-2:

- 4 Process (P1, P2, P3, P4) ve 3 Kaynak (R1, R2, R3) içeren bir Kaynak Tahsis Grafiği düşünün. Wait-for-Graph tabanlı kilitlenme algılama algoritmasını kullanarak bir kilitlenme olup olmadığını bulunuz.





- Resource Allocation Graph

wait-for-graph

- Wait-for-Graph üzerinde bir döngü varmış gibi görünse de aslında kapalı bir döngü yoktur. Kilitlenme oluşmaz.

Kilitlenmeyi Algılama Algoritması (Deadlock Detection)

- *Bir sistemde **n*** process ve ***m*** farklı kaynak türü mevcut ise aşağıdaki vektörler ve matrisler tanımlanabilir:

- Kaynaklar **(Resource)** = $\mathbf{R} = (R_1, R_2, \ldots, R_m)$

- Boşta bulunan kaynaklar **(Available)** = $\mathbf{V} = (V_1, V_2, \ldots, V_m)$

- Kaynak istekleri **(Request)** = **Q** $= \begin{bmatrix} Q_{11} & Q_{12} & \ldots & Q_{1m} \\ Q_{21} & Q_{22} & \ldots & Q_{2m} \\ . & . & & . \\ Q_{n1} & Q_{n2} & \ldots & Q_{nm} \end{bmatrix}$

- Atanmalar **(Allocation)** = **A** $= \begin{bmatrix} A_{11} & A_{12} & \ldots & A_{1m} \\ A_{21} & A_{22} & \ldots & A_{2m} \\ . & . & & . \\ A_{n1} & A_{n2} & \ldots & A_{nm} \end{bmatrix}$

Kilitlenmeyi Algılama (Deadlock Detection)

- Kilitlenmeyi Algılama Yöntemi aşağıdaki gibidir:

1. *Bir sistemde **n*** adet process ve ***m*** farklı kaynak türü mevcut ise ***m*** uzunluğunda Work (W) vektörü ve ***n*** uzunluğunda Finish (F) vektörü tanımlanır.
2. Başlangıçta Work (W) vektörü, Available (V) vektörüne eşitlenmelidir (**Work= Available**).
3. i=0, 1, …., n-1 için ,

   Allocation[i] = 0 ise, Finish[i] = true; **Allocation[i] ≠ 0 ise, Finish[i] = false**.
4. **a) Finish[i] == false** ve **b) Request_i <= Work** olacak şekilde bir process bulunmaya çalışılır.
5. Eğer böyle bir process varsa;

   Work= Work + Allocation[i] ve **Finish[i]= true** → **Kilitlenme yok**

   işlemleri yapılır. Ardından 4. adıma dönülmelidir.
6. 4. adımdaki durumu sağlayacak bir process yoksa;

   Eğer Finish[i]== false ise, sistem kilitlenme durumundadır.

   Finish[i]==false ise, Pi process'i kilitlenir. → **Kilitlenme var!**

Kilitlenmeyi Algılama Algoritması (Deadlock Detection)

	R1	R2	…..	Rm
P**0**				
P**1**				
P**2**				
…..				
P**n-1**				

i = 0, 1, …., n-1

Örnek :

- Aşağıdaki bilgileri kullanarak kilitlenme (deadlock) olup olmadığını tespit ediniz.

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Örnek :

- **Work = Available**
- Work = [0, 0, 0]

- **i=0, 1, …., n-1 için , Allocation[i] = 0 ise, Finish[i] = true;** **Allocation[i] ≠ 0 ise, Finish[i] = false.**
- Finish = [false, false, false, false, false] (çünkü tüm process'ler başlangıçta kaynak kullanıyor.)

- **Finish[i] == false ve Request[i] <= Work olacak şekilde bir process bulunmaya çalışılır.**

Örnek :

- **Finish[i] == false ve Request[i] <= Work**
- Şartlarını sağlayan process P0
- i=**0, P0** process'i için:
- Finish[**0**] = false ve [0, 0, 0]<=[0, 0, 0] şartları sağlanıyor.

- **Work= Work + Allocation[0]** ve **Finish[0]= true**
- Work =[0, 0, 0]+[0, 1, 0] =>[0, 1, 0]

- Finish = [**true**, false, false, false, false].

Örnek :

- **Finish[i] == false ve Request[i] <= Work**
- Şartlarını sağlayan process P2
- i=**2,** P2 process'i için :
- Finish[**2**] = false ve [0, 0, 0]<=[0, 1, 0] şartları sağlanıyor.

- **Work= Work + Allocation[2]** ve **Finish[2]= true**
- Work =[0, 1, 0]+[3, 0, 3] =>[3, 1, 3]
- Finish = [true, false, **true**, false, false].

Örnek :

- **Finish[i] == false ve Request[i] <= Work**
- Şartlarını sağlayan process P1
- i=**1,** P1 process'i için :

- Finish[**1**] = false ve [2, 0, 2]<=[3, 1, 3] şartları sağlanıyor.

- **Work= Work + Allocation[1]** ve **Finish[1]= true**
- Work =[3, 1, 3]+[2, 0, 0] =>[5, 1, 3]
- Finish = [true, **true**, true, false, false]

Örnek :

- **Finish[i] == false ve Request[i] <= Work**
- Şartlarını sağlayan process P1
- i=**3,** P3 process'i için :

- Finish[**3**] = false ve [1, 0, 0]<=[5, 1, 3] şartları sağlanıyor.

- **Work= Work + Allocation[3]** ve **Finish[3]= true**
- Work =[5, 1, 3]+[2, 1, 1] =>[7, 2, 4]
- Finish = [true, true, true, **true**, false].

Örnek :

- **Finish[i] == false ve Request[i] <= Work**
- Şartlarını sağlayan process P4
- i=**4,** P4 process'i için :
- Finish[**4**] = false ve [0, 0, 2]<=[7, 2, 4] şartları sağlanıyor.

- **Work= Work + Allocation[4]** ve **Finish[4]= true**
- Work =[7, 2, 4]+[0, 0, 2] =>[7, 2, 6]
- Finish = [true, true, true, true, **true**].

- Finish, vektörünün tüm elemanları **true** olduğundan, bu örnekte bir kilitlenme olmadığı anlamına gelir.

Kilitlenme Kurtarma (Deadlock Recovery)

- Eğer kilitlenme tespit edilirse, sistem şu yöntemlerden biriyle kurtarılır:

- **A) Process Sonlandırma (Process Termination)**
- Tüm kilitlenmiş process'leri sonlandır (ağır ama kesin çözüm).
- Tek tek process'leri sonlandırarak döngüyü kır (en az zararlı seçenek seçilir).

- **B) Kaynak Geri Alma (Resource Preemption)**
- Bir process'in kaynakları zorla alınır ve diğer process'lere verilir.
- Risk: Aynı process sürekli mağdur edilebilir (starvation).